Midterm Project Report

Traditional Image Processing for Filtering, 3D Reconstruction, and Image Stitching

Name: Ha Manh Duc
Student ID: 23070287
Email: 23070287@vnu.edu.vn
Course: INS3155 – Computer Vision

November 2026

Abstract

This report implements and systematically compares classical (non-learning) computer vision methods on three tasks built from a custom architectural photograph. In **Part A**, Gaussian ($\sigma_n = 25$) and salt-and-pepper (8%) noise are removed with Mean, Gaussian, and Median filters and enhanced with Laplacian sharpening over 3×3, 5×5 and 7×7 kernels; the 3×3 median filter restores salt-and-pepper images from 15.83 dB to 26.81 dB PSNR (SSIM 0.928), while a small Gaussian kernel is best for Gaussian noise (24.77 dB). In **Part B**, the fundamental matrix is estimated with SIFT and RANSAC (Sampson error 0.24 px), and SGBM reduces disparity RMSE by about 43% compared to Block Matching against a known ground truth. In **Part C**, a four-view panorama pipeline compares SIFT, ORB and AKAZE: SIFT and AKAZE reach sub-0.27 px reprojection RMSE, while ORB is roughly four times less accurate. Laplacian pyramid blending and parameter sensitivity studies are included as extensions.

Contents

1 Introduction **3**

2 Implementation and Experimental Setup **3**

3 Part A: Image Filtering **4**
3.1 Methodology 4
3.2 Implementation and Results 4
3.3 Comparative Analysis 7

4 Part B: 3D Reconstruction from Stereo Images **9**
4.1 Methodology 9
4.2 Implementation and Results 10
4.3 Comparative Analysis 12

5 Part C: Multi-View Image Stitching **13**

5.1 Methodology 13
5.2 Implementation and Results 14
5.3 Comparative Analysis 16

6 Conclusion and Future Work **18**

References **18**

A Appendix: Python Code **19**
A.1 Part A: Image Filtering 19
A.2 Part B: 3D Reconstruction 21
A.3 Part C: Image Stitching 26

1 Introduction

Classical image processing and multi-view geometry remain the backbone of many vision systems, from camera image signal processors and document scanners to visual odometry and smartphone panorama modes. Unlike deep networks, these methods are analytically interpretable, need no training data, and run deterministically on a CPU, which makes them easy to reason about and to benchmark. This project studies three such building blocks:

- **Image filtering (Part A):** denoising is the first stage of almost every pipeline; the key question is how to suppress noise without destroying edges.
- **Stereo 3D reconstruction (Part B):** recovering depth from two views underpins robotics, autonomous driving and photogrammetry; it requires epipolar geometry and dense correspondence.
- **Image stitching (Part C):** building panoramas tests feature detection, robust matching, homography estimation and blending in one end-to-end system.

For each task at least two methods or parameter settings are compared quantitatively (PSNR, SSIM, valid-pixel ratio, RMSE, inlier ratio, runtime) and qualitatively. Section 2 describes the data and environment, Sections 3–5 present each part (methodology, results, comparative analysis), and Section 6 concludes.

2 Implementation and Experimental Setup

Dataset. All experiments start from a custom photograph of a decorated architectural wall (`data/IMG_0891.jpg`, 1024×768), which contains strong repetitive geometric patterns, fine ornaments, a window frame and textureless plaster regions. From it we derive:

- **Part A:** a 640×480 grayscale clean image, corrupted with additive Gaussian noise ($\mu = 0, \sigma_n = 25$) and salt-and-pepper noise (4% salt + 4% pepper). The random seed is fixed (42) for reproducibility.
- **Part B:** a rectified stereo pair with a *known ground-truth disparity*. The scene is modelled as a slanted wall (disparity $14 \rightarrow 40$ px), a foreground pillar (44 px) and a raised elliptical relief (+10 px). The right view is synthesized as $I_R(x, y) = I_L(x + d(x, y), y)$, and the ground truth is forward-mapped to the left image grid, leaving occluded pixels undefined (7.3% of the image). This allows exact RMSE and bad-pixel evaluation, which is impossible with uncalibrated handheld photographs.
- **Part C:** four overlapping 480×560 views with ≈50% horizontal overlap, each with a different small perspective warp and a different exposure gain (1.00, 0.88, 1.10, 0.94) to mimic handheld capture with auto-exposure.

Environment. Python 3.13, OpenCV 4.14 (`opencv-python<5`, since AKAZE is not available in the OpenCV 5 main package), NumPy, SciPy, scikit-image (metrics), pandas and Matplotlib on an Apple Silicon MacBook (macOS 14). Runtimes are wall-clock times measured with `time.perf_counter()` on a single run and should be read as relative rather than absolute.

Reproducibility. Each part is a standalone script (`part_a_filtering.py`, `part_b_stereo_3d.py`, `part_c_stitching.py`) that regenerates every figure and CSV in `results/`. All tables in this report are generated from these CSV files by `generate_report_tables.py`, so the report always matches the code output.

3 Part A: Image Filtering

3.1 Methodology

Noise models. Gaussian noise adds $\tilde{I} = \text{clip}(I + n)$ with $n \sim \mathcal{N}(0, \sigma_n^2)$; salt-and-pepper noise sets a random fraction p_s of pixels to 255 and p_p to 0 (impulse noise).

Mean (box) filter. Every pixel is replaced by the average of its $k \times k$ neighbourhood Ω:

$$\hat{I}(x, y) = \frac{1}{k^2} \sum_{(i,j)\in\Omega} \tilde{I}(x + i, y + j). \tag{1}$$

Gaussian filter. A weighted average with a separable kernel

$$G_\sigma(i, j) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right), \qquad \hat{I} = G_\sigma * \tilde{I}. \tag{2}$$

For the kernel-size study we use $\sigma = 0.8, 1.2, 1.6$ for $k = 3, 5, 7$; a separate sweep varies $\sigma \in [0.5, 3]$ with kernel size $k = 2\lceil 3\sigma \rceil + 1$ so the kernel is never truncated.

Median filter. A non-linear order statistic, $\hat{I}(x, y) = \text{median}\{\tilde{I}(x + i, y + j) \mid (i, j) \in \Omega\}$, which rejects outliers instead of averaging them.

Laplacian sharpening. Using the discrete Laplacian $\nabla^2 I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$ (OpenCV aperture k),

$$I_{\text{sharp}} = I_{\text{den}} - \alpha \, \nabla^2 I_{\text{den}}, \tag{3}$$

where I_{den} is the Gaussian-filtered (Gaussian noise) or median-filtered (impulse noise) image of the same kernel size. Because the Laplacian response grows quickly with aperture size, α is reduced to 0.25, 0.12 and 0.03 for $k = 3, 5, 7$.

Metrics. MSE $= \frac{1}{N}\sum(I - \hat{I})^2$, PSNR $= 10 \log_{10}(255^2/\text{MSE})$ and SSIM (luminance, contrast and structure similarity over local windows), all computed against the clean image, plus runtime in ms.

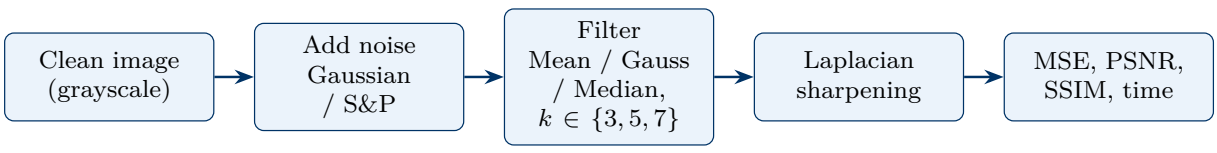

Figure 1: Part A workflow.

3.2 Implementation and Results

The filters use `cv2.blur`, `cv2.GaussianBlur`, `cv2.medianBlur` and `cv2.Laplacian`; metrics use `skimage.metrics`. Figures 2 and 3 show the original, the noisy input and all 12

filter outputs per noise type. The noisy inputs score 20.61 dB / SSIM 0.469 (Gaussian) and 15.83 dB / SSIM 0.390 (S&P).

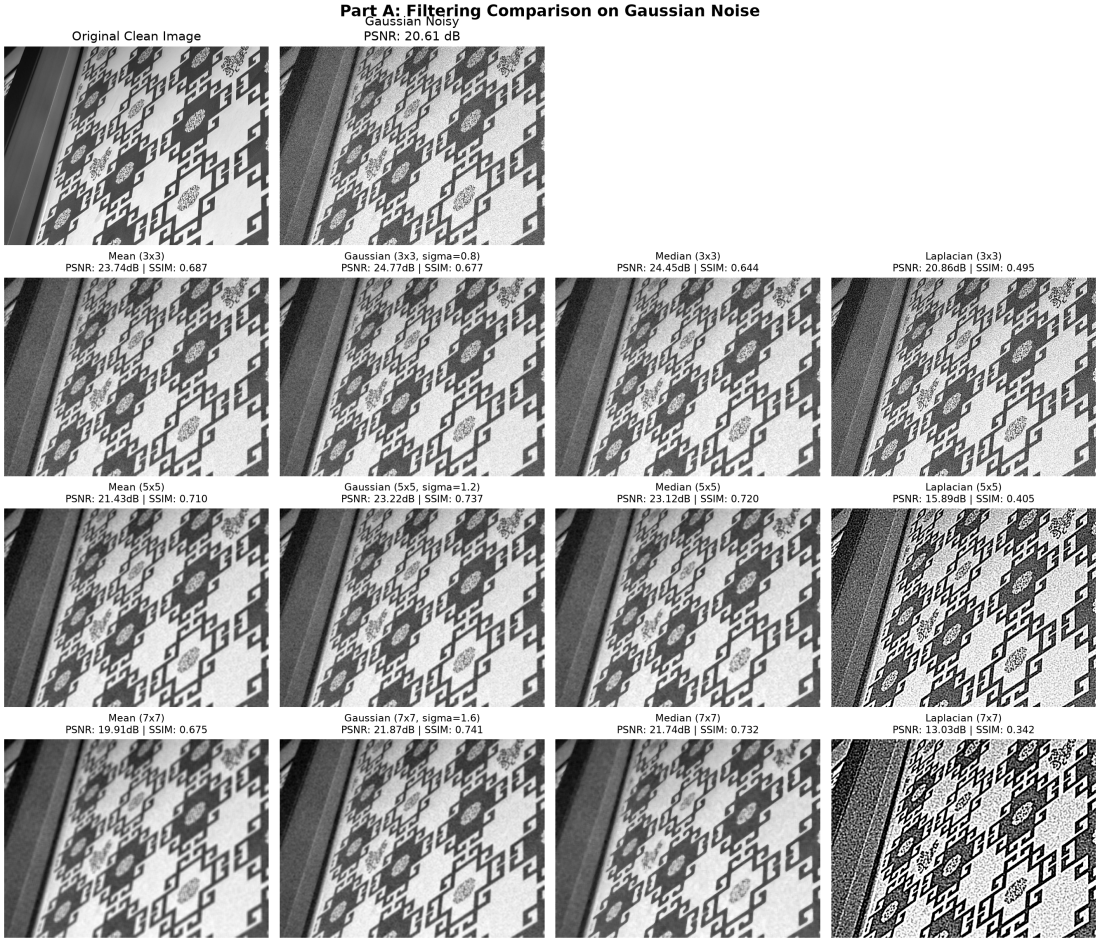

Figure 2: Filtering results on Gaussian noise ($\sigma_n = 25$). Rows: kernel 3×3, 5×5, 7×7.

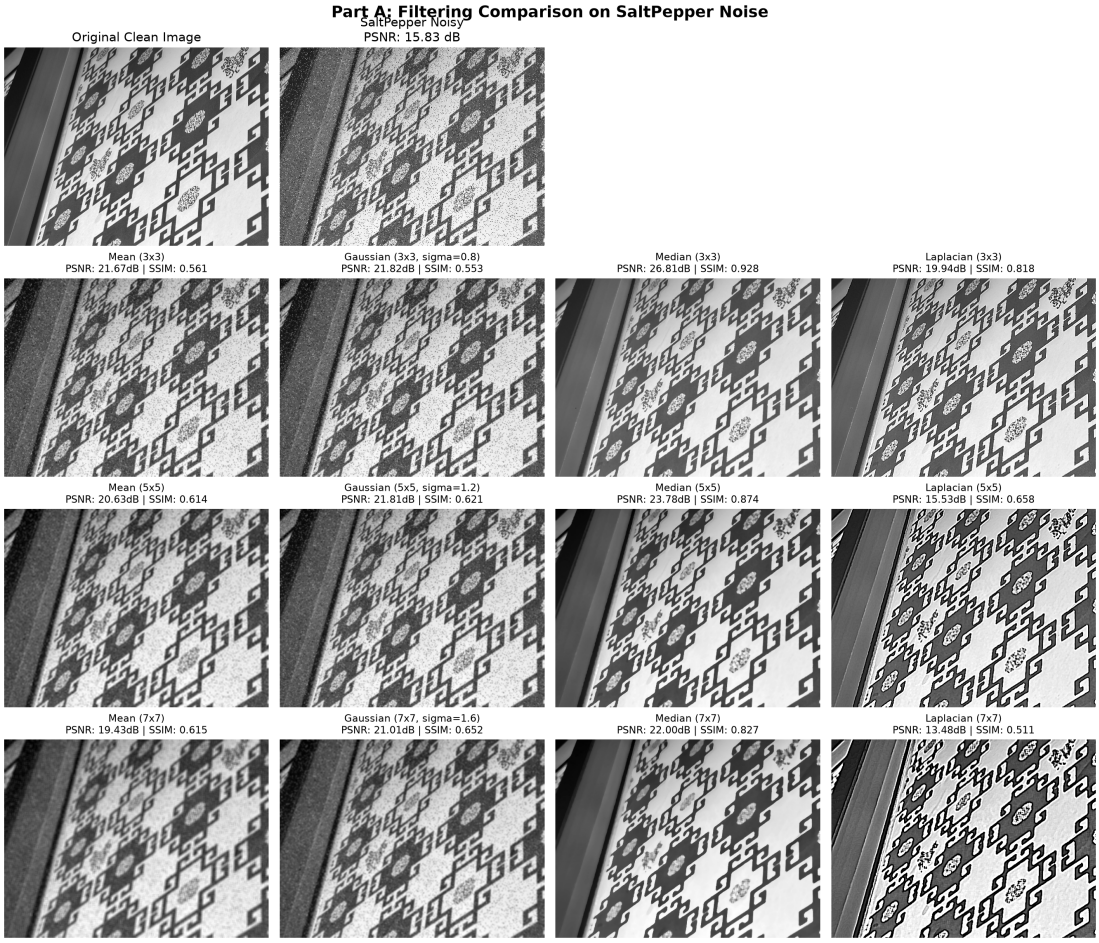

Figure 3: Filtering results on salt-and-pepper noise (8%).

3.3 Comparative Analysis

Table 1: Quantitative comparison of spatial filters (best PSNR per noise type in bold).

Noise	Filter	Kernel	MSE ↓	PSNR (dB) ↑	SSIM ↑	Time (ms) ↓
Gaussian (σ_n=25)	Mean	3×3	275.13	23.74	0.6870	1.389
Gaussian (σ_n=25)	**Gaussian (σ=0.8)**	**3×3**	**216.71**	**24.77**	**0.6768**	**1.163**
Gaussian (σ_n=25)	Median	3×3	233.37	24.45	0.6439	0.439
Gaussian (σ_n=25)	Laplacian	3×3	533.73	20.86	0.4946	1.949
Gaussian (σ_n=25)	Mean	5×5	468.23	21.43	0.7102	0.092
Gaussian (σ_n=25)	Gaussian (σ=1.2)	5×5	309.70	23.22	0.7370	0.352
Gaussian (σ_n=25)	Median	5×5	316.77	23.12	0.7198	0.157
Gaussian (σ_n=25)	Laplacian	5×5	1675.13	15.89	0.4055	2.810
Gaussian (σ_n=25)	Mean	7×7	663.66	19.91	0.6748	0.234
Gaussian (σ_n=25)	Gaussian (σ=1.6)	7×7	423.12	21.87	0.7408	0.371
Gaussian (σ_n=25)	Median	7×7	436.06	21.74	0.7321	0.509
Gaussian (σ_n=25)	Laplacian	7×7	3237.72	13.03	0.3421	1.848
S&P (8%)	Mean	3×3	443.03	21.67	0.5606	0.138
S&P (8%)	Gaussian (σ=0.8)	3×3	427.20	21.82	0.5530	0.187
S&P (8%)	**Median**	**3×3**	**135.57**	**26.81**	**0.9284**	**0.105**
S&P (8%)	Laplacian	3×3	659.73	19.94	0.8179	2.021
S&P (8%)	Mean	5×5	562.61	20.63	0.6138	0.146
S&P (8%)	Gaussian (σ=1.2)	5×5	428.76	21.81	0.6208	0.237
S&P (8%)	Median	5×5	272.33	23.78	0.8740	0.205
S&P (8%)	Laplacian	5×5	1818.02	15.53	0.6582	1.987
S&P (8%)	Mean	7×7	741.87	19.43	0.6154	0.371
S&P (8%)	Gaussian (σ=1.6)	7×7	515.76	21.01	0.6523	0.300
S&P (8%)	Median	7×7	410.55	22.00	0.8271	0.455
S&P (8%)	Laplacian	7×7	2920.78	13.48	0.5108	3.335

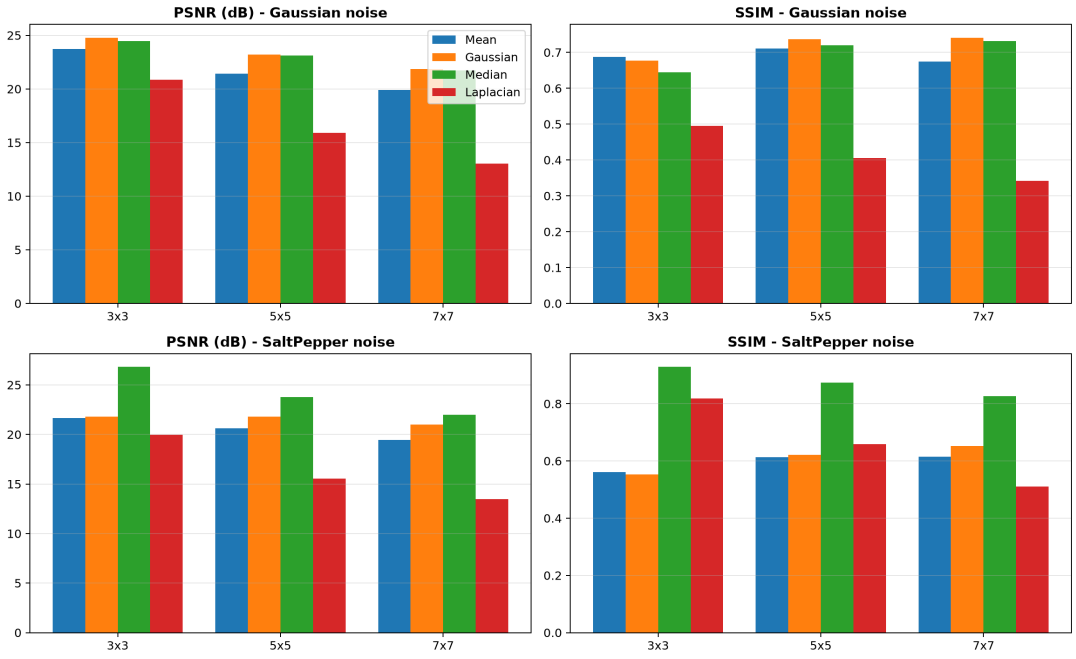

Figure 4: PSNR and SSIM per filter and kernel size for both noise types.

Table 2: Gaussian filter σ sensitivity (best PSNR and best SSIM per noise type in bold).

Noise	σ	Kernel	MSE	PSNR (dB)	SSIM	Time (ms)
Gaussian	**0.5**	**5×5**	**271.73**	**23.79**	**0.5636**	**0.250**
Gaussian	1.0	7×7	272.76	23.77	0.7269	0.489
Gaussian	**1.5**	**11×11**	**415.00**	**21.95**	**0.7420**	**0.594**
Gaussian	2.0	13×13	568.51	20.58	0.7159	0.730
Gaussian	2.5	17×17	727.07	19.52	0.6756	0.875
Gaussian	3.0	19×19	879.84	18.69	0.6363	1.224
S&P	0.5	5×5	770.51	19.26	0.4692	0.532
S&P	**1.0**	**7×7**	**410.13**	**22.00**	**0.6039**	**0.563**
S&P	1.5	11×11	509.63	21.06	0.6515	0.774
S&P	**2.0**	**13×13**	**650.67**	**20.00**	**0.6579**	**0.878**
S&P	2.5	17×17	803.27	19.08	0.6397	0.785
S&P	3.0	19×19	952.60	18.34	0.6133	0.902

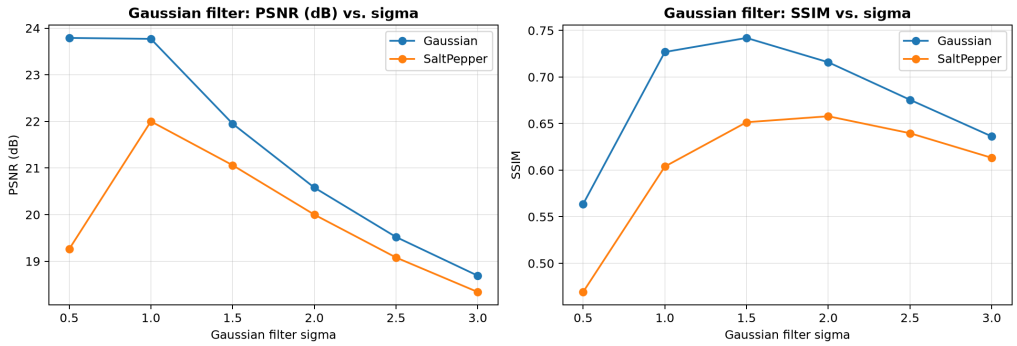

Figure 5: PSNR and SSIM as a function of the Gaussian filter standard deviation.

Quantitative analysis.

- **Salt-and-pepper noise:** the 3×3 median filter is the clear winner (26.81 dB, SSIM 0.928), about 5 dB above the best linear filter. Mean and Gaussian filters only spread each impulse over the kernel, so their SSIM stays at 0.55–0.65 for every kernel size and every σ (Table 2).
- **Gaussian noise:** PSNR is highest for small kernels (Gaussian 3×3, 24.77 dB; median 3×3, 24.45 dB), whereas SSIM peaks with moderate smoothing (Gaussian 7×7, 0.741; $\sigma = 1.5$, 0.742). Stronger smoothing removes more noise, which helps local structure, but also blurs the fine ornaments, which costs pixel-wise error.
- **Kernel size:** for every filter, PSNR decreases monotonically from 3×3 to 7×7. The mean filter degrades fastest (23.74 → 19.91 dB) because its flat weights blur most; the Gaussian filter degrades slowest.
- σ **sweep:** beyond $\sigma \approx 1.5$ both PSNR and SSIM drop steadily for both noise types, so large σ is never beneficial on this detailed image.
- **Runtime:** the denoising filters take only 0.1–0.5 ms on a 640×480 image (the very first call, ≈1.4 ms, includes OpenCV warm-up), and the optimized median filter is as

fast as the linear ones. Laplacian sharpening is the slowest step (1.8–3.3 ms) because it runs in 32-bit floating point and must be converted back to 8-bit.

Qualitative analysis.

- **Noise removal vs. edge preservation:** the median filter keeps the step edges of the geometric pattern sharp, while mean and Gaussian filters produce soft, blurred edges at 7×7 (over-smoothing). On Gaussian noise, however, the median filter leaves a blotchy, "painted" texture in flat plaster regions.
- **Linear filters on impulses:** with S&P noise, mean and Gaussian filters turn each impulse into a grey blob that is still clearly visible, which explains their low SSIM.
- **Laplacian sharpening:** at 3×3 it visibly crisps edges of the ornaments and window frame, but because it is a high-pass operation it also amplifies the residual noise and produces thin halo/overshoot bands along strong edges. With larger apertures the halos widen and the PSNR falls to 13–16 dB, even though the image looks sharper. Sharpening therefore should only be applied after effective denoising and with a small aperture; PSNR/SSIM against the clean image penalise it by design.

4 Part B: 3D Reconstruction from Stereo Images

4.1 Methodology

Epipolar geometry. Corresponding points $\mathbf{x} \leftrightarrow \mathbf{x}'$ (homogeneous) in the left and right images satisfy

$$\mathbf{x}'^{\top} F\, \mathbf{x} = 0, \qquad \mathbf{l}' = F\mathbf{x}, \quad \mathbf{l} = F^{\top}\mathbf{x}', \tag{4}$$

where F is the 3×3 rank-2 fundamental matrix and $\mathbf{l}'$ is the epipolar line of $\mathbf{x}$ in the right image. We detect SIFT keypoints, match them with a brute-force KNN matcher ($k = 2$) and Lowe's ratio test (0.75), and estimate F with RANSAC (`cv2.findFundamentalMat`, threshold 1.5 px, confidence 0.99). Two errors are reported over the RANSAC inliers: the symmetric point-to-epipolar-line distance $d(\mathbf{x}, \mathbf{l}) = |\mathbf{l}^{\top}\mathbf{x}| / \sqrt{l_1^2 + l_2^2}$ (averaged over both images) and the first-order geometric (Sampson) error

$$e_S = \frac{|\mathbf{x}'^{\top} F \mathbf{x}|}{\sqrt{(F\mathbf{x})_1^2 + (F\mathbf{x})_2^2 + (F^{\top}\mathbf{x}')_1^2 + (F^{\top}\mathbf{x}')_2^2}}. \tag{5}$$

Dense disparity. For a rectified pair, a left pixel (x, y) matches $(x - d, y)$ in the right image.

- **Block Matching (BM)** chooses, independently per pixel, the disparity minimizing the sum of absolute differences over a $b \times b$ window: $d^*(p) = \arg\min_d \sum_{q \in W_p} |I_L(q) - I_R(q - d)|$.
- **Semi-Global Matching (SGBM)** minimizes a global energy that adds smoothness penalties, approximated by dynamic programming along 8 directions (3-way mode):

$$E(D) = \sum_p C(p, D_p) + \sum_{q \in N_p} P_1 \left[|D_p - D_q| = 1\right] + \sum_{q \in N_p} P_2 \left[|D_p - D_q| > 1\right], \tag{6}$$

with $P_1 = 8 \cdot 3 \cdot b^2$ and $P_2 = 32 \cdot 3 \cdot b^2$, uniqueness ratio 10 and speckle filtering.

Depth and point cloud. With focal length f and baseline B, depth is $Z = fB/d$. We use `cv2.reprojectImageTo3D` with

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & -1 & 0 & c_y \\ 0 & 0 & 0 & -f \\ 0 & 0 & 1/B & 0 \end{bmatrix}, \qquad [X\ Y\ Z\ W]^\top = Q\,[x\ y\ d\ 1]^\top, \tag{7}$$

assuming $f = 0.8w = 512\,\text{px}$, $B = 0.12\,\text{m}$ and the principal point at the image centre.

Metrics. Valid disparity ratio (% of pixels with $d > 0$), RMSE and bad-pixel rate ($|d - d_{gt}| > 2\,\text{px}$) against the ground truth, evaluated on non-occluded pixels with $x \geq 128$ (the left band where no full disparity search is possible is excluded identically for all settings), and runtime.

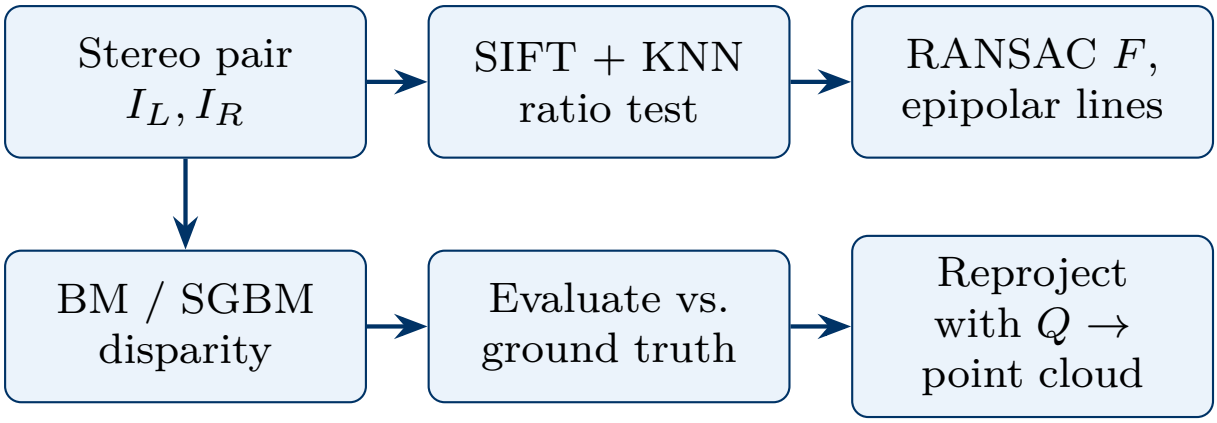

Figure 6: Part B workflow.

4.2 Implementation and Results

Figure 7 shows 30 inlier correspondences and their epipolar lines. As expected for a rectified pair, the lines are almost horizontal and each point lies on the line of its partner. Figure 8 compares the ground truth with five BM/SGBM settings, and Figure 9 shows the reconstructed point cloud (68 223 coloured points, depth 1.39–3.43 m, exported to `results/part_b/reconstructed_cloud.ply`).

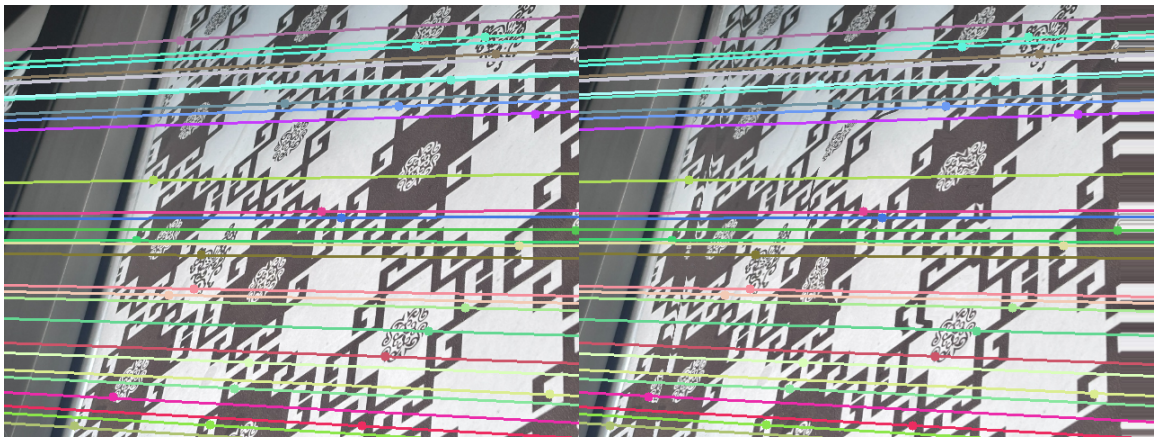

Figure 7: Epipolar lines and RANSAC inlier points on the left (left half) and right (right half) images.

Table 3: Fundamental matrix estimation (SIFT + RANSAC).

Ratio-test matches	RANSAC inliers	Inlier ratio (%)	Epipolar err. (px)	Sampson err. (px)
1277	1010	79.09	0.3388	0.2395

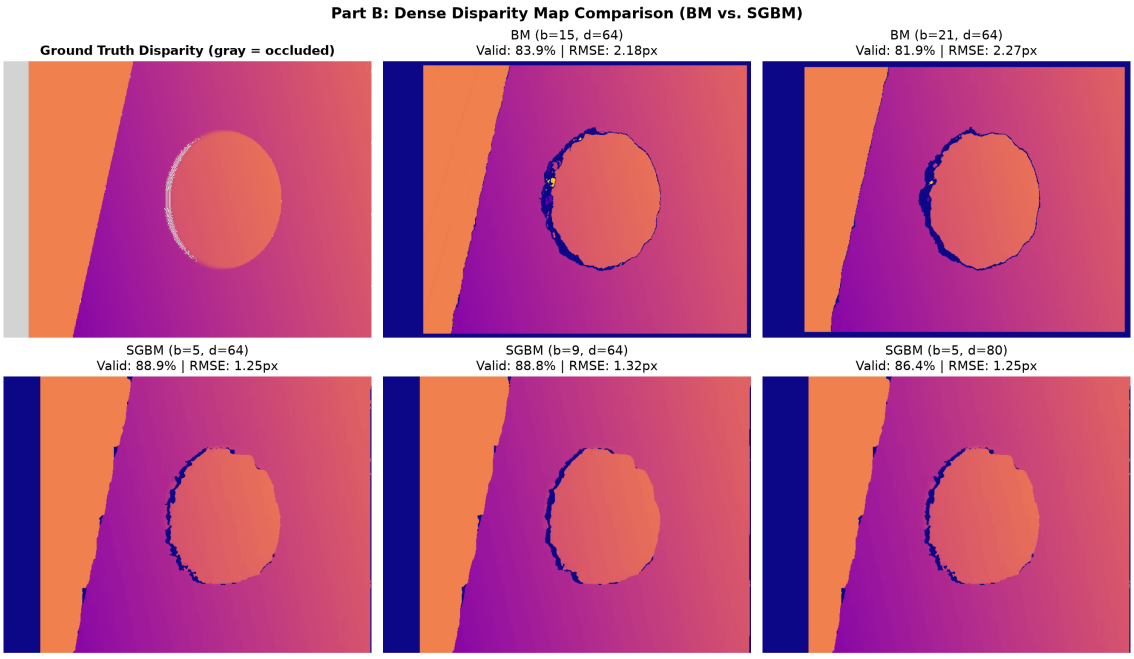

Figure 8: Ground truth and colourized disparity maps for BM and SGBM (b = blockSize, d = numDisparities). Dark blue = invalid.

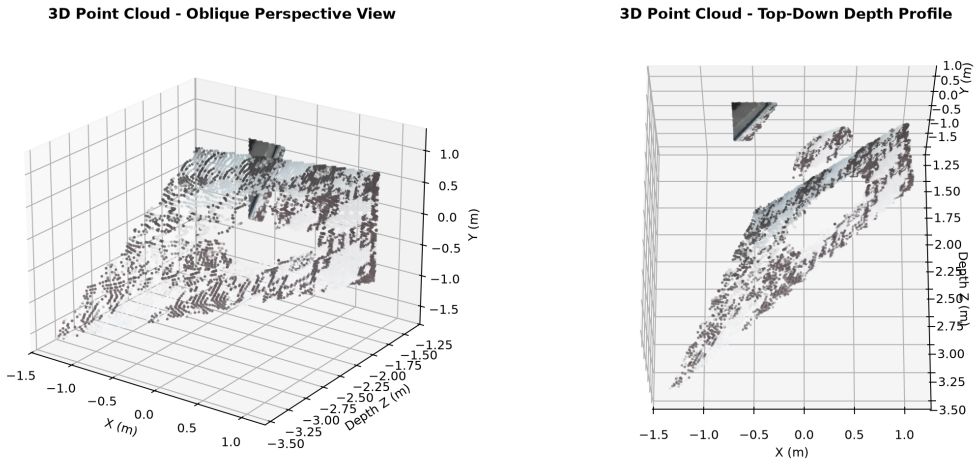

Figure 9: Two views of the 3D point cloud reconstructed from SGBM ($b = 9$, $d = 64$).

4.3 Comparative Analysis

Table 4: BM vs. SGBM (best RMSE in bold).

Method	blockSize	numDisp.	Valid (%) ↑	RMSE (px) ↓	Bad >2px (%) ↓	Runtime (ms) ↓
BM	15	64	83.94	2.18	1.16	8.7
BM	21	64	81.95	2.27	1.32	8.9
SGBM	**5**	**64**	**88.91**	**1.25**	**1.09**	**19.5**
SGBM	9	64	88.83	1.32	1.16	23.7
SGBM	5	80	86.40	1.25	1.09	26.1

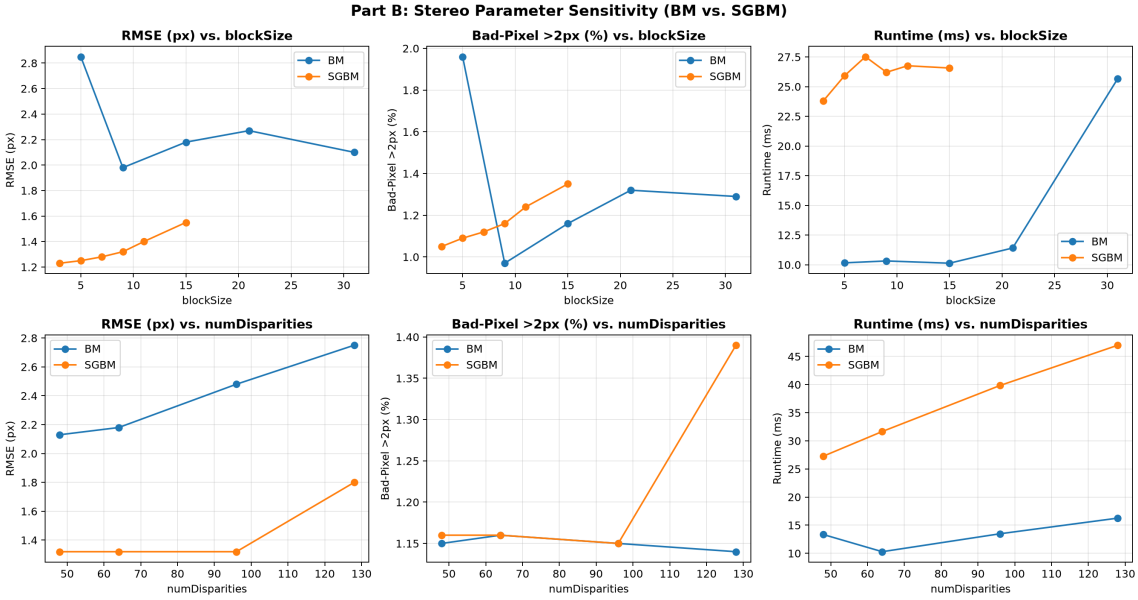

Figure 10: Sensitivity of BM and SGBM to blockSize (top, numDisparities = 64) and numDisparities (bottom).

Quantitative analysis.

- **Epipolar geometry:** 79% of the 1 277 ratio-test matches survive RANSAC, with a mean epipolar distance of 0.34 px and Sampson error of 0.24 px, i.e. sub-pixel consistency, so the estimated F is reliable.
- **BM vs. SGBM:** at the same numDisparities, SGBM produces about 5 percentage points more valid pixels (88.9% vs. 83.9%) and roughly 43% lower RMSE (1.25 vs. 2.18 px). The bad-pixel rate is low for both (≈1.1%), meaning BM's extra error is spread as small inaccuracies rather than gross mismatches. SGBM is about 2–3 times slower (19.5 vs. 8.7 ms) because it aggregates costs along several paths, and its runtime grows linearly with numDisparities (27 → 47 ms from 48 to 128 levels), whereas BM stays around 10–16 ms.
- **blockSize:** BM is unstable with small windows ($b = 5$: RMSE 2.85 px, 1.96% bad pixels) and is best around $b = 9$; larger windows lose valid pixels (78.7% at $b = 31$). SGBM is much less sensitive thanks to its smoothness term; its error grows slowly from 1.23 px ($b = 3$) to 1.55 px ($b = 15$) as windows start to straddle depth edges.

- **numDisparities:** the valid ratio falls linearly with numDisparities because the left-most numDisparities columns cannot be matched. Since the maximum ground-truth disparity is 44 px, 48 levels are already sufficient; 128 levels increase ambiguity (SGBM RMSE 1.80 px) and runtime without any benefit.

Qualitative analysis.

- **Smoothness:** the wall is densely textured, so both methods recover the slanted disparity ramp smoothly; this also explains the low bad-pixel rates. BM additionally loses a frame of about half a block width on every image border.
- **Occlusions:** in the occluded strip on the left of the elliptical relief (grey in the ground truth) no correspondence exists. BM still assigns disparities there, producing blobs of grossly wrong (too large) values, whereas SGBM's uniqueness check mostly marks these pixels invalid, which is the desired behaviour.
- **Object boundaries:** BM "fattens" the foreground pillar into the background by roughly half a window because the SAD window straddles the depth edge. SGBM keeps the boundary closer to the ground truth but shows small jagged streaks along it, a typical artefact of path-wise cost aggregation.
- **3D geometry:** the point cloud correctly separates the pillar (nearest, ≈1.4 m), the raised relief, and the slanted wall receding to ≈3.4 m. Depth quantization (disparity steps of 1/16 px) is visible as slight layering in the top-down view.

5 Part C: Multi-View Image Stitching

5.1 Methodology

Feature extraction.

- **SIFT:** Difference-of-Gaussian extrema in scale space, sub-pixel refined, with a 128-D gradient-orientation histogram descriptor (L2 distance).
- **ORB:** oriented FAST corners with rotated BRIEF 256-bit binary descriptors (Hamming distance), limited to 3 000 features per image.
- **AKAZE:** keypoints in a non-linear diffusion scale space, which preserves edges better than Gaussian blurring, with binary M-LDB descriptors (Hamming distance).

Matching. Brute-force KNN ($k = 2$) with Lowe's ratio test: a match m is kept if $\|f - f_1\| < 0.75\,\|f - f_2\|$, where f_1, f_2 are the nearest and second-nearest descriptors.

Homography. For a planar scene or pure camera rotation, corresponding points satisfy

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}, \tag{8}$$

with 8 degrees of freedom, estimated by the DLT inside RANSAC with a reprojection threshold of 2 or 4 px. Accuracy is measured as the inlier reprojection RMSE $\sqrt{\frac{1}{N}\sum_i \|\mathbf{x}_i - \pi(H\mathbf{x}'_i)\|^2}$.

Warping. View 2 is the reference frame. Homographies are chained as $H_{1\to2}$, I, $H_{3\to2}$ and $H_{4\to2} = H_{3\to2}H_{4\to3}$; all images are warped with an extra translation so that the panorama has non-negative coordinates.

Blending.

- **Feathering:** weights are proportional to the Euclidean distance transform of each valid mask, $w_k = D_k/(D_1 + D_2)$, so each image fades out towards its border.
- **Laplacian pyramid (multi-band):** each image is decomposed into band-pass levels $L_\ell = G_\ell - \mathrm{up}(G_{\ell+1})$, and each level is blended with a Gaussian-blurred version of the binary seam mask M_ℓ: $L_\ell^{\text{out}} = (1 - M_\ell)L_\ell^A + M_\ell L_\ell^B$, then the pyramid is collapsed. Low frequencies (exposure) are mixed over a wide region while high frequencies (edges) switch over a narrow one, which avoids both visible seams and ghosting.
- **None (overwrite)** is used as a baseline.

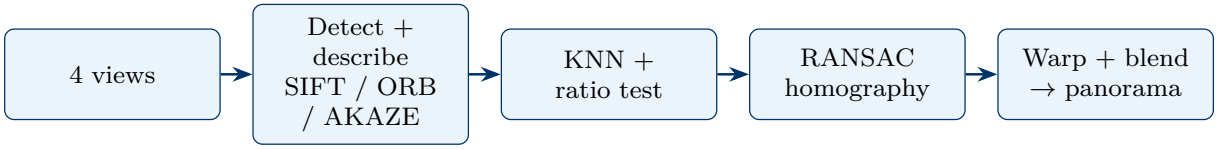

Figure 11: Part C workflow.

5.2 Implementation and Results

The pipeline is built from the following OpenCV functions:

- detection: `SIFT_create`, `ORB_create`, `AKAZE_create`;
- matching and geometry: `BFMatcher.knnMatch`, `findHomography` (RANSAC), `warpPerspective`;
- blending (with NumPy): `distanceTransform`, `pyrDown`, `pyrUp`.

Figure 12 shows the input views, Figure 13 the detected keypoints, and Figure 14 the inlier matches between views 2 and 3 with the final panoramas.

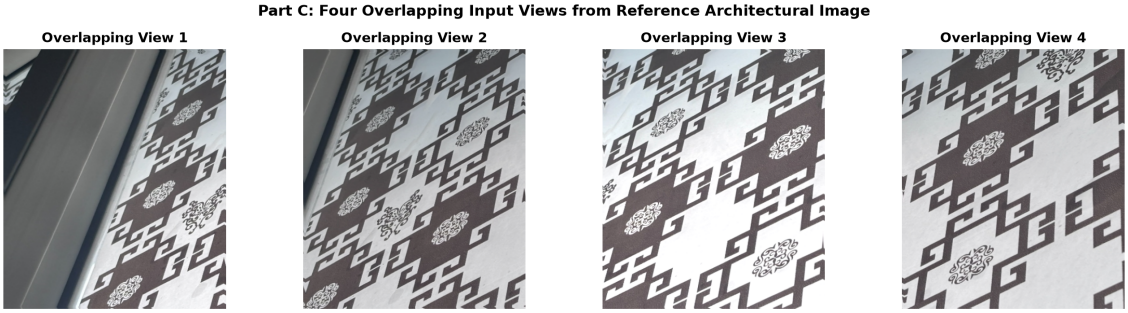

Figure 12: The four overlapping input views (different perspective and exposure).

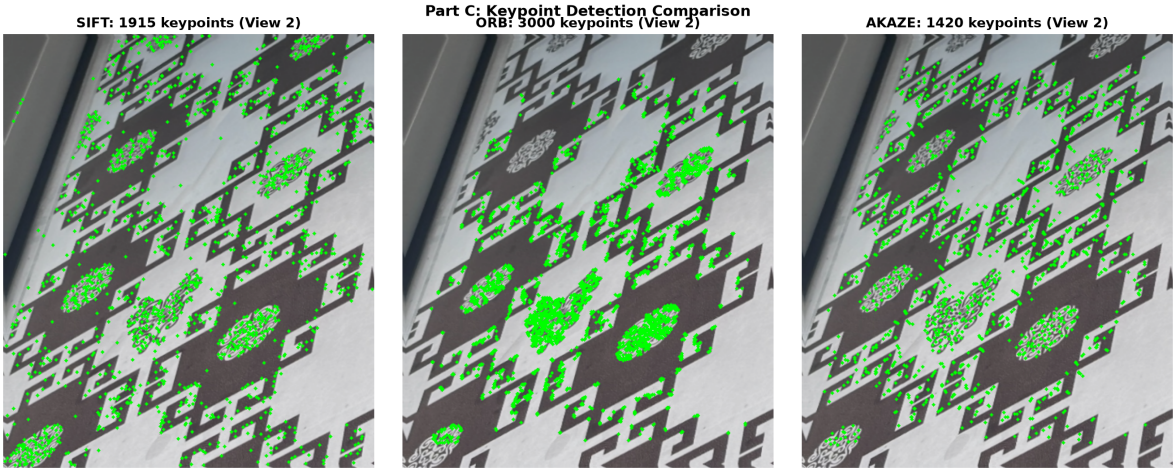

Figure 13: Keypoints detected on view 2 by SIFT, ORB and AKAZE.

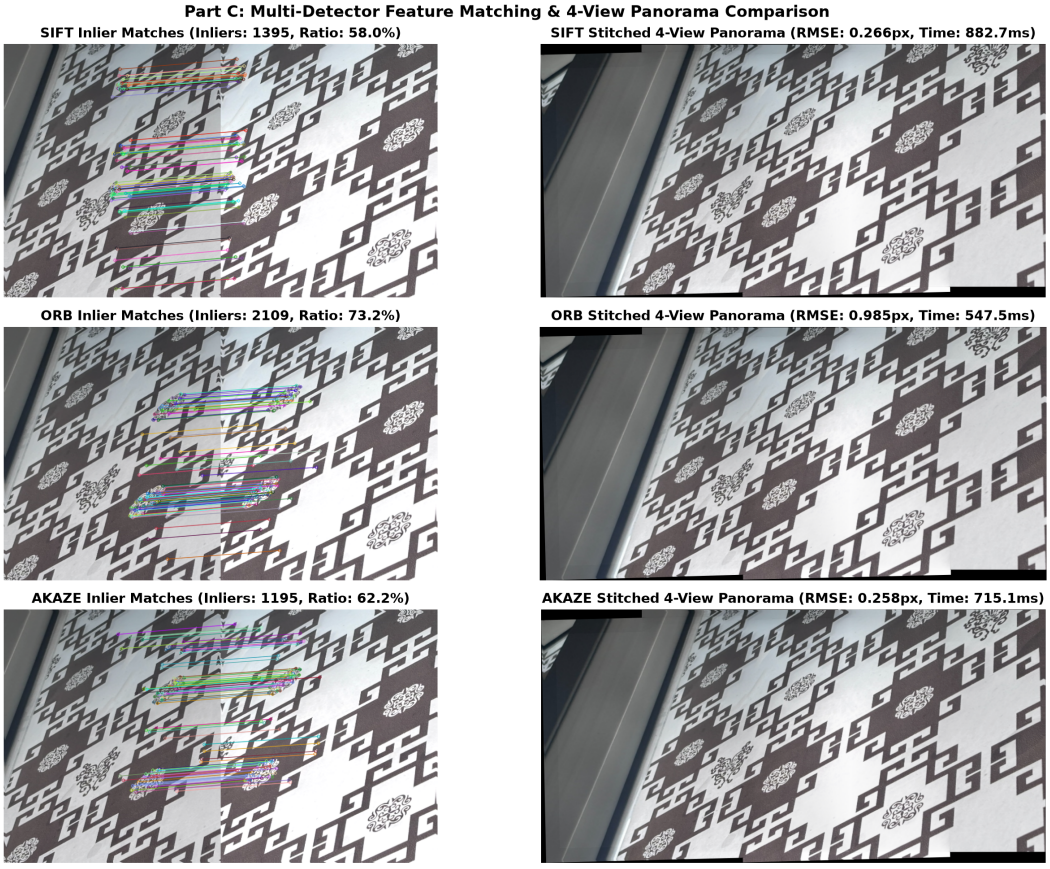

Figure 14: RANSAC inlier matches (views 2–3, first 80 shown) and 4-view panoramas (feather blending, RANSAC 4 px).

5.3 Comparative Analysis

Table 5: Detector comparison summed over the three image pairs (best RMSE in bold). Inlier ratio and RMSE are averaged over the pairs.

Detector	RANSAC (px)	Keypoints	Matches	Inliers	Inlier (%)	RMSE (px)	Time (ms)
SIFT	**2.0**	**9976**	**2394**	**1392**	**57.84**	**0.232**	**813.4**
SIFT	4.0	9976	2394	1395	57.95	0.266	882.7
ORB	2.0	17593	2873	2010	69.79	0.874	781.0
ORB	4.0	17593	2873	2109	73.23	0.985	547.5
AKAZE	4.0	7167	1912	1195	62.16	0.258	715.1

Table 6: Blending comparison (SIFT, RANSAC 4 px).

Blending method	Blending time (ms)	Total time (ms)
None (overwrite)	155.8	594.3
Feather (distance transform)	381.6	1149.3
Laplacian pyramid (5 levels)	598.4	1286.9

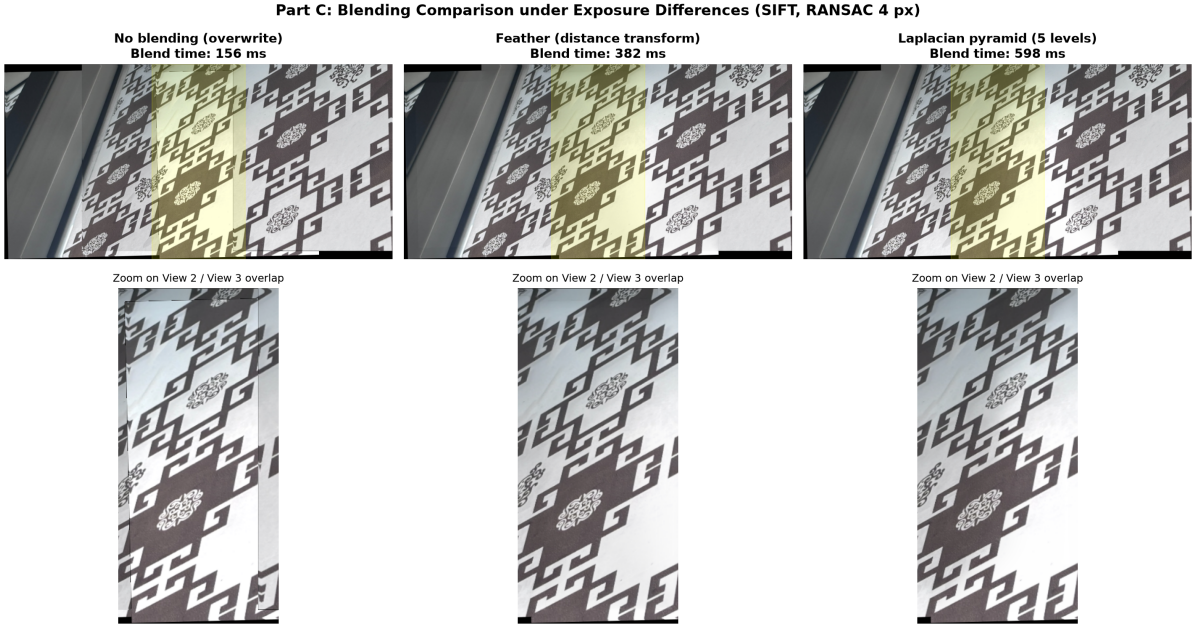

Figure 15: Panoramas without blending, with feathering and with Laplacian pyramid blending; bottom row zooms on the view 2/3 overlap where the exposure jumps from 0.88 to 1.10.

Quantitative analysis.

- **Keypoints and matches:** the keypoint column sums both images of all three pairs (six detections). ORB returns the most keypoints (≈2 930 per image, close to its cap of 3 000) and the most matches and inliers; SIFT finds ≈1 660 and AKAZE the fewest (≈1 190 per image), but AKAZE keeps a comparable inlier ratio. As Figure 13 shows,

ORB keeps only the 3 000 strongest Harris-ranked corners, which cluster densely on the high-contrast ornaments in the centre and leave the image periphery almost empty; SIFT and AKAZE spread their keypoints much more evenly over the pattern corners and the whole view, which gives better-conditioned homography estimates.

- **Accuracy:** SIFT (0.23–0.27 px) and AKAZE (0.26 px) are about four times more accurate than ORB (0.87–0.99 px). SIFT and AKAZE localize keypoints with sub-pixel refinement in a scale space, while ORB uses integer FAST corners on a coarse pyramid.
- **Inlier ratio:** 58–73% for all detectors. The strongly repetitive wall pattern produces many plausible but wrong matches that the ratio test cannot reject, which RANSAC then removes.
- **RANSAC threshold:** raising the threshold from 2 to 4 px adds inliers (ORB +99, or +3.4 points of inlier ratio) but also raises RMSE, since less accurate correspondences are accepted. For accurate detectors like SIFT the threshold hardly matters (+3 inliers).
- **Runtime:** SIFT is the slowest pipeline (813–883 ms) because of its floating-point scale space and 128-D L2 matching; ORB (547–781 ms) and AKAZE (715 ms) are faster thanks to binary descriptors and Hamming distance. Warping and blending of the panorama canvas take 280–430 ms of the total, so the end-to-end difference between detectors is smaller than the difference in feature extraction alone. Single-run timings vary by roughly ±20%, so only the ordering should be relied on.

Qualitative analysis.

- **Geometric alignment:** all three detectors give globally consistent panoramas, and even under zoom the pattern edges in the overlaps show no visible doubling (ghosting). ORB's higher RMSE (≈1 px) is below what is visible on this scene, so for this planar scene all three are adequate; the difference would matter for larger images or longer homography chains where errors accumulate.
- **Seams and illumination:** without blending, the exposure changes produce hard vertical seams and visible frame borders (Figure 15). Feathering removes the seams but spreads the exposure change over the whole overlap, and any misalignment shows as ghosting. Laplacian pyramid blending gives the most natural transition: brightness is equalized smoothly while edges remain sharp, at about 1.6 times the blending cost of feathering (Table 6).
- **Perspective distortion:** the wedge-shaped black borders at the panorama corners show how each view's perspective warp is undone when it is mapped into the plane of view 2. Views 1 and 4 are reached through one and two chained homographies, so any error in $H_{4\to3}$ propagates into $H_{4\to2}$. For wider panoramas, a cylindrical projection or bundle adjustment would be needed to limit stretching at the ends.

6 Conclusion and Future Work

The experiments confirm that the best classical method depends on the noise or scene model. For impulse noise, the non-linear median filter is far better than any linear filter, whereas for Gaussian noise small Gaussian kernels give the best PSNR and moderate smoothing gives the best SSIM; Laplacian sharpening improves perceived sharpness but must follow denoising. In stereo, SIFT + RANSAC yields a sub-pixel fundamental matrix, and SGBM's global smoothness term produces denser and about 43% more accurate disparity than local Block Matching for a 2–3× runtime cost, while being much less sensitive to block size. In stitching, SIFT and AKAZE give the most accurate homographies, ORB is the fastest but least precise, and Laplacian pyramid blending best hides exposure differences.

Limitations and future work. The stereo pair and multi-view images are synthesized from one photograph so that exact ground truth is available; the next step is to capture a calibrated real stereo pair (e.g. with `cv2.stereoCalibrate` and rectification) and to evaluate on the Middlebury benchmark. Other improvements include edge-preserving filters (bilateral, non-local means), left-right consistency checks and WLS filtering for disparity maps, gain compensation and bundle adjustment for stitching, and seam finding via graph cuts.

References

[1] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022.

[2] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.

[3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE TIP*, 13(4), 2004.

[5] H. Hirschmüller, "Stereo processing by semiglobal matching and mutual information," *IEEE TPAMI*, 30(2), 2008.

[6] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *IJCV*, 60(2), 2004.

[7] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: an efficient alternative to SIFT or SURF," *ICCV*, 2011.

[8] P. F. Alcantarilla, J. Nuevo and A. Bartoli, "Fast explicit diffusion for accelerated features in nonlinear scale spaces," *BMVC*, 2013.

[9] M. A. Fischler and R. C. Bolles, "Random sample consensus," *Communications of the ACM*, 24(6), 1981.

[10] P. J. Burt and E. H. Adelson, "A multiresolution spline with application to image mosaics," *ACM TOG*, 2(4), 1983.

[11] OpenCV documentation, https://docs.opencv.org/4.x/.

A Appendix: Python Code

Full source code and results: https://github.com/Habuck/computer-vision.

A.1 Part A: Image Filtering

```python
import os
import time
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
from skimage.metrics import mean_squared_error as mse

os.makedirs("results/part_a", exist_ok=True)
os.makedirs("report_latex/figures", exist_ok=True)

def add_gaussian_noise(image, mean=0, sigma=25):
    gauss = np.random.normal(mean, sigma, image.shape)
    noisy = np.clip(image.astype(np.float32) + gauss, 0, 255)
    return noisy.astype(np.uint8)

def add_salt_pepper_noise(image, salt_prob=0.04, pepper_prob=0.04):
    noisy = image.copy()
    rand = np.random.rand(*image.shape)
    noisy[rand < salt_prob] = 255
    noisy[(rand >= salt_prob) & (rand < salt_prob + pepper_prob)] = 0
    return noisy

def laplacian_sharpening(image, ksize=3, alpha=0.25):
    lap = cv2.Laplacian(image, cv2.CV_32F, ksize=ksize)
    sharp = np.clip(image.astype(np.float32) - alpha * lap, 0, 255)
    return sharp.astype(np.uint8)

def run_part_a(img_path="data/reference_image.jpg"):
    clean_bgr = cv2.imread(img_path)
    clean = cv2.cvtColor(clean_bgr, cv2.COLOR_BGR2GRAY)
    clean = cv2.resize(clean, (640, 480))
    cv2.imwrite("data/clean_gray.png", clean)

    np.random.seed(42)
    noises = {
        "Gaussian": add_gaussian_noise(clean, sigma=25),
        "SaltPepper": add_salt_pepper_noise(clean, 0.04, 0.04)
    }

    kernel_sizes = [3, 5, 7]
    records = []

    for noise_name, noisy_img in noises.items():
        cv2.imwrite(f"results/part_a/noisy_{noise_name}.png", noisy_img)
        fig, axes = plt.subplots(4, 4, figsize=(15, 13))
        fig.suptitle(f"Part A: Filtering Comparison on {noise_name} Noise", fontsize=15,
            fontweight='bold')

        axes[0, 0].imshow(clean, cmap='gray')
        axes[0, 0].set_title("Original Clean Image")
        axes[0, 1].imshow(noisy_img, cmap='gray')
        axes[0, 1].set_title(f"{noise_name} Noisy\nPSNR: {psnr(clean, noisy_img,
            data_range=255):.2f} dB")
        axes[0, 2].axis('off')
        axes[0, 3].axis('off')
        axes[0, 0].axis('off')
        axes[0, 1].axis('off')

        for idx, k in enumerate(kernel_sizes):
            # 1. Mean Filter
            t0 = time.perf_counter()
            mean_img = cv2.blur(noisy_img, (k, k))
```

```
            t_mean = (time.perf_counter() - t0) * 1000

            # 2. Gaussian Filter
            sigma = round(0.8 + 0.4 * idx, 1)
            t0 = time.perf_counter()
            gauss_img = cv2.GaussianBlur(noisy_img, (k, k), sigmaX=sigma)
            t_gauss = (time.perf_counter() - t0) * 1000

            # 3. Median Filter
            t0 = time.perf_counter()
            med_img = cv2.medianBlur(noisy_img, k)
            t_med = (time.perf_counter() - t0) * 1000

            # 4. Laplacian Sharpening (on denoised image to enhance architectural edges)
            base_denoised = gauss_img if noise_name == "Gaussian" else med_img
            alpha_val = 0.25 if k == 3 else (0.12 if k == 5 else 0.03)
            t0 = time.perf_counter()
            sharp_img = laplacian_sharpening(base_denoised, ksize=k, alpha=alpha_val)
            t_sharp = (time.perf_counter() - t0) * 1000

            outputs = [
                (f"Mean ({k}x{k})", mean_img, t_mean),
                (f"Gaussian ({k}x{k}, sigma={sigma})", gauss_img, t_gauss),
                (f"Median ({k}x{k})", med_img, t_med),
                (f"Laplacian ({k}x{k})", sharp_img, t_sharp)
            ]

            for col_idx, (f_name, out_img, exec_t) in enumerate(outputs):
                val_mse = mse(clean, out_img)
                val_psnr = psnr(clean, out_img, data_range=255)
                val_ssim = ssim(clean, out_img, data_range=255)

                records.append({
                    "Noise Type": noise_name,
                    "Filter": f_name,
                    "Kernel": f"{k}x{k}",
                    "MSE": round(val_mse, 2),
                    "PSNR (dB)": round(val_psnr, 2),
                    "SSIM": round(val_ssim, 4),
                    "Time (ms)": round(exec_t, 3)
                })

                ax = axes[idx + 1, col_idx]
                ax.imshow(out_img, cmap='gray')
                ax.set_title(f"{f_name}\nPSNR: {val_psnr:.2f}dB | SSIM: {val_ssim:.3f}",
                    fontsize=9.5)
                ax.axis('off')

        plt.tight_layout()
        out_fig = f"results/part_a/grid_{noise_name}.png"
        plt.savefig(out_fig, dpi=180)
        plt.savefig(f"report_latex/figures/grid_{noise_name}.png", dpi=180)
        plt.close()

    df = pd.DataFrame(records)
    df.to_csv("results/part_a/quantitative_part_a.csv", index=False)
    print(df.to_string(index=False))

    plot_metric_bars(df)
    sigma_sensitivity(clean, noises)

def plot_metric_bars(df):
    filters = ["Mean", "Gaussian", "Median", "Laplacian"]
    kernels = ["3x3", "5x5", "7x7"]
    width = 0.2
    x = np.arange(len(kernels))

    fig, axes = plt.subplots(2, 2, figsize=(13, 8))
    for row, noise_name in enumerate(["Gaussian", "SaltPepper"]):
        sub = df[df["Noise Type"] == noise_name]
        for col, metric in enumerate(["PSNR (dB)", "SSIM"]):
            ax = axes[row, col]
            for i, f in enumerate(filters):
```

```
                vals = [sub[(sub["Filter"].str.startswith(f)) & (sub["Kernel"] ==
                    k)][metric].values[0] for k in kernels]
                ax.bar(x + (i - 1.5) * width, vals, width, label=f)
            ax.set_xticks(x)
            ax.set_xticklabels(kernels)
            ax.set_title(f"{metric} - {noise_name} noise", fontweight='bold')
            ax.grid(axis='y', alpha=0.3)
            if row == 0 and col == 0:
                ax.legend()
    plt.tight_layout()
    plt.savefig("results/part_a/metrics_bar_chart.png", dpi=180)
    plt.savefig("report_latex/figures/metrics_bar_chart.png", dpi=180)
    plt.close()

def sigma_sensitivity(clean, noises, sigmas=(0.5, 1.0, 1.5, 2.0, 2.5, 3.0)):
    records = []
    for noise_name, noisy_img in noises.items():
        for s in sigmas:
            k = 2 * int(np.ceil(3 * s)) + 1
            t0 = time.perf_counter()
            out = cv2.GaussianBlur(noisy_img, (k, k), sigmaX=s)
            exec_t = (time.perf_counter() - t0) * 1000
            records.append({
                "Noise Type": noise_name,
                "sigma": s,
                "Kernel": f"{k}x{k}",
                "MSE": round(mse(clean, out), 2),
                "PSNR (dB)": round(psnr(clean, out, data_range=255), 2),
                "SSIM": round(ssim(clean, out, data_range=255), 4),
                "Time (ms)": round(exec_t, 3)
            })

    df = pd.DataFrame(records)
    df.to_csv("results/part_a/sigma_sensitivity.csv", index=False)
    print(df.to_string(index=False))

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
    for noise_name in noises:
        sub = df[df["Noise Type"] == noise_name]
        axes[0].plot(sub["sigma"], sub["PSNR (dB)"], marker='o', label=noise_name)
        axes[1].plot(sub["sigma"], sub["SSIM"], marker='o', label=noise_name)
    for ax, metric in zip(axes, ["PSNR (dB)", "SSIM"]):
        ax.set_xlabel("Gaussian filter sigma")
        ax.set_ylabel(metric)
        ax.set_title(f"Gaussian filter: {metric} vs. sigma", fontweight='bold')
        ax.grid(alpha=0.3)
        ax.legend()
    plt.tight_layout()
    plt.savefig("results/part_a/sigma_sensitivity.png", dpi=180)
    plt.savefig("report_latex/figures/sigma_sensitivity.png", dpi=180)
    plt.close()

if __name__ == "__main__":
    run_part_a()
```

A.2 Part B: 3D Reconstruction

```
import os
import time
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

os.makedirs("results/part_b", exist_ok=True)
os.makedirs("report_latex/figures", exist_ok=True)

def create_stereo_from_reference(ref_path="data/reference_image.jpg"):
    img = cv2.imread(ref_path)
    img = cv2.resize(img, (640, 480))
    h, w = img.shape[:2]
```

```
15
16      # Build a realistic 3D depth/disparity ground-truth map for the architectural wall:
17      # Slanted decorative wall (disparity increases from left=14px to right=36px) + Foreground
            structural column on left (disparity=46px)
18      xx, yy = np.meshgrid(np.arange(w, dtype=np.float32), np.arange(h, dtype=np.float32))
19      gt_disp = 14.0 + 22.0 * (xx / w) + 4.0 * ((h - yy) / h)
20
21      # Left window frame / pillar in foreground (approx x < 170 - 0.18*y)
22      pillar_mask = xx < (185 - 0.22 * yy)
23      gt_disp[pillar_mask] = 44.0
24
25      # Central raised geometric emblem relief
26      relief_mask = ((xx - 350)**2 / 95**2 + (yy - 240)**2 / 120**2) <= 1.0
27      gt_disp[relief_mask] += 10.0
28
29      gt_disp = cv2.GaussianBlur(gt_disp, (9, 9), 2.0)
30
31      # Synthesize right stereo view via backward-compatible horizontal disparity remap: I_R(x
            - d, y) = I_L(x, y)
32      map_x = (xx + gt_disp).astype(np.float32)
33      map_y = yy.astype(np.float32)
34      imgL = img.copy()
35      # In stereo, a point at (x, y) in Left image appears at (x - d, y) in Right image ->
            I_R(x, y) = I_L(x + d, y)
36      imgR = cv2.remap(imgL, map_x, map_y, interpolation=cv2.INTER_LINEAR,
            borderMode=cv2.BORDER_REPLICATE)
37
38      cv2.imwrite("data/stereo_left.png", imgL)
39      cv2.imwrite("data/stereo_right.png", imgR)
40      return imgL, imgR, disparity_right_to_left(gt_disp)
41
42  def disparity_right_to_left(gt_disp_r):
43      # gt_disp_r is indexed by right-image pixels (x_L = x_R + d), but BM/SGBM report
            disparity on the left grid.
44      # Forward-splat to the left grid; nearer surfaces (larger d) win, unfilled pixels are
            occlusions (NaN).
45      h, w = gt_disp_r.shape
46      gt_left = np.full((h, w), np.nan, dtype=np.float32)
47      ys, xs = np.mgrid[0:h, 0:w]
48      xl = np.round(xs + gt_disp_r).astype(int)
49      order = np.argsort(gt_disp_r, axis=None)
50      ys, xl, d = ys.ravel()[order], xl.ravel()[order], gt_disp_r.ravel()[order]
51      inside = xl < w
52      gt_left[ys[inside], xl[inside]] = d[inside]
53
54      left, right = np.roll(gt_left, 1, axis=1), np.roll(gt_left, -1, axis=1)
55      pinhole = np.isnan(gt_left) & ~np.isnan(left) & ~np.isnan(right) & (np.abs(left - right)
            < 1.5)
56      gt_left[pinhole] = 0.5 * (left[pinhole] + right[pinhole])
57      return gt_left
58
59  def compute_epipolar_geometry(imgL, imgR):
60      grayL = cv2.cvtColor(imgL, cv2.COLOR_BGR2GRAY)
61      grayR = cv2.cvtColor(imgR, cv2.COLOR_BGR2GRAY)
62
63      sift = cv2.SIFT_create()
64      kp1, des1 = sift.detectAndCompute(grayL, None)
65      kp2, des2 = sift.detectAndCompute(grayR, None)
66
67      bf = cv2.BFMatcher()
68      matches = bf.knnMatch(des1, des2, k=2)
69      pts1, pts2 = [], []
70      for m, n in matches:
71          if m.distance < 0.75 * n.distance:
72              pts1.append(kp1[m.queryIdx].pt)
73              pts2.append(kp2[m.trainIdx].pt)
74
75      pts1 = np.float32(pts1)
76      pts2 = np.float32(pts2)
77
78      F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC, 1.5, 0.99)
79      inliers1 = pts1[mask.ravel() == 1]
80      inliers2 = pts2[mask.ravel() == 1]
```

```
81
82      lines1 = cv2.computeCorrespondEpilines(inliers2.reshape(-1, 1, 2), 2, F).reshape(-1, 3)
83      lines2 = cv2.computeCorrespondEpilines(inliers1.reshape(-1, 1, 2), 1, F).reshape(-1, 3)
84
85      err1 = np.abs(lines1[:, 0]*inliers1[:, 0] + lines1[:, 1]*inliers1[:, 1] + lines1[:, 2]) /
            np.hypot(lines1[:, 0], lines1[:, 1])
86      err2 = np.abs(lines2[:, 0]*inliers2[:, 0] + lines2[:, 1]*inliers2[:, 1] + lines2[:, 2]) /
            np.hypot(lines2[:, 0], lines2[:, 1])
87      mean_epi_err = float(np.mean(err1 + err2) / 2.0)
88      mean_sampson = sampson_error(F, inliers1, inliers2)
89
90      visL, visR = imgL.copy(), imgR.copy()
91      w = imgL.shape[1]
92      np.random.seed(12)
93      indices = np.linspace(0, len(inliers1) - 1, min(30, len(inliers1)), dtype=int)
94      for idx in indices:
95          r1, r2, pt1, pt2 = lines1[idx], lines2[idx], inliers1[idx], inliers2[idx]
96          color = tuple(np.random.randint(40, 255, 3).tolist())
97          x0, y0 = map(int, [0, -r1[2]/r1[1]])
98          x1, y1 = map(int, [w, -(r1[2]+r1[0]*w)/r1[1]])
99          cv2.line(visL, (x0, y0), (x1, y1), color, 2)
100         cv2.circle(visL, tuple(np.int32(pt1)), 5, color, -1)
101
102         x0, y0 = map(int, [0, -r2[2]/r2[1]])
103         x1, y1 = map(int, [w, -(r2[2]+r2[0]*w)/r2[1]])
104         cv2.line(visR, (x0, y0), (x1, y1), color, 2)
105         cv2.circle(visR, tuple(np.int32(pt2)), 5, color, -1)
106
107     epi_pair = np.hstack([visL, visR])
108     cv2.imwrite("results/part_b/epipolar_lines.png", epi_pair)
109     cv2.imwrite("report_latex/figures/epipolar_lines.png", epi_pair)
110     return F, len(pts1), len(inliers1), mean_epi_err, mean_sampson
111
112 def sampson_error(F, pts1, pts2):
113     x1 = np.hstack([pts1, np.ones((len(pts1), 1))])
114     x2 = np.hstack([pts2, np.ones((len(pts2), 1))])
115     Fx1 = x1 @ F.T
116     Ftx2 = x2 @ F
117     num = np.sum(x2 * Fx1, axis=1) ** 2
118     den = Fx1[:, 0]**2 + Fx1[:, 1]**2 + Ftx2[:, 0]**2 + Ftx2[:, 1]**2
119     return float(np.mean(np.sqrt(num / den)))
120
121 def compute_disparity(grayL, grayR, algo, bsize, ndisp):
122     t0 = time.perf_counter()
123     if algo == "BM":
124         matcher = cv2.StereoBM_create(numDisparities=ndisp, blockSize=bsize)
125     else:
126         matcher = cv2.StereoSGBM_create(
127             minDisparity=0,
128             numDisparities=ndisp,
129             blockSize=bsize,
130             P1=8 * 3 * bsize**2,
131             P2=32 * 3 * bsize**2,
132             uniquenessRatio=10,
133             speckleWindowSize=100,
134             speckleRange=32,
135             mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
136         )
137     disp = matcher.compute(grayL, grayR).astype(np.float32) / 16.0
138     return disp, (time.perf_counter() - t0) * 1000
139
140 def evaluate_disparity(disp, gt_disp, margin):
141     valid_mask = disp > 0
142     valid_ratio = np.sum(valid_mask) / disp.size * 100.0
143     # Skip the left band where no full disparity search is possible and occluded pixels
            without ground truth.
144     eval_mask = valid_mask & ~np.isnan(gt_disp) & (np.arange(disp.shape[1])[None, :] >=
            margin)
145     err = disp[eval_mask] - gt_disp[eval_mask]
146     rmse = float(np.sqrt(np.mean(err**2)))
147     bad = float(np.mean(np.abs(err) > 2.0) * 100.0)
148     return valid_ratio, rmse, bad
149
```

```python
def parameter_sensitivity(grayL, grayR, gt_disp, margin):
    records = []
    sweeps = [
        ("BM", "blockSize", [(b, 64) for b in [5, 9, 15, 21, 31]]),
        ("SGBM", "blockSize", [(b, 64) for b in [3, 5, 7, 9, 11, 15]]),
        ("BM", "numDisparities", [(15, n) for n in [48, 64, 96, 128]]),
        ("SGBM", "numDisparities", [(9, n) for n in [48, 64, 96, 128]]),
    ]
    for algo, varied, settings in sweeps:
        for bsize, ndisp in settings:
            disp, runtime = compute_disparity(grayL, grayR, algo, bsize, ndisp)
            valid, rmse, bad = evaluate_disparity(disp, gt_disp, margin)
            records.append({
                "Method": algo, "Varied": varied, "blockSize": bsize, "numDisparities": ndisp,
                "Valid Disparity (%)": round(valid, 2), "RMSE (px)": round(rmse, 2),
                "Bad-Pixel >2px (%)": round(bad, 2), "Runtime (ms)": round(runtime, 2)
            })

    df = pd.DataFrame(records)
    df.to_csv("results/part_b/parameter_sensitivity.csv", index=False)
    print(df.to_string(index=False))

    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    for row, varied in enumerate(["blockSize", "numDisparities"]):
        for col, metric in enumerate(["RMSE (px)", "Bad-Pixel >2px (%)", "Runtime (ms)"]):
            ax = axes[row, col]
            for algo in ["BM", "SGBM"]:
                sub = df[(df["Method"] == algo) & (df["Varied"] == varied)]
                ax.plot(sub[varied], sub[metric], marker='o', label=algo)
            ax.set_xlabel(varied)
            ax.set_ylabel(metric)
            ax.set_title(f"{metric} vs. {varied}", fontweight='bold')
            ax.grid(alpha=0.3)
            ax.legend()
    plt.suptitle("Part B: Stereo Parameter Sensitivity (BM vs. SGBM)", fontsize=14,
        fontweight='bold')
    plt.tight_layout()
    plt.savefig("results/part_b/parameter_sensitivity.png", dpi=180)
    plt.savefig("report_latex/figures/parameter_sensitivity.png", dpi=180)
    plt.close()

def write_ply(filename, verts, colors):
    verts = verts.reshape(-1, 3)
    colors = colors.reshape(-1, 3)
    ply_header = f"""ply
format ascii 1.0
element vertex {len(verts)}
property float x
property float y
property float z
property uchar red
property uchar green
property uchar blue
end_header
"""
    with open(filename, 'w') as f:
        f.write(ply_header)
        for v, c in zip(verts, colors):
            f.write(f"{v[0]:.4f} {v[1]:.4f} {v[2]:.4f} {int(c[0])} {int(c[1])} {int(c[2])}\n")

def run_part_b():
    imgL, imgR, gt_disp = create_stereo_from_reference()
    grayL = cv2.cvtColor(imgL, cv2.COLOR_BGR2GRAY)
    grayR = cv2.cvtColor(imgR, cv2.COLOR_BGR2GRAY)

    F, total_matches, num_inliers, epi_err, sampson = compute_epipolar_geometry(imgL, imgR)
    print(f"Stereo Matches: {total_matches}, Inliers: {num_inliers}, "
          f"Mean Epipolar Error: {epi_err:.4f} px, Mean Sampson Error: {sampson:.4f} px")
    pd.DataFrame([{
        "Ratio-test Matches": total_matches,
        "RANSAC Inliers": num_inliers,
        "Inlier Ratio (%)": round(num_inliers / total_matches * 100.0, 2),
        "Mean Epipolar Error (px)": round(epi_err, 4),
```

```
        "Mean Sampson Error (px)": round(sampson, 4)
    }]).to_csv("results/part_b/fundamental_matrix.csv", index=False)
    np.savetxt("results/part_b/fundamental_matrix.txt", F, fmt="%.8e")

    eval_margin = 128

    configs = [
        ("BM", 15, 64),
        ("BM", 21, 64),
        ("SGBM", 5, 64),
        ("SGBM", 9, 64),
        ("SGBM", 5, 80)
    ]

    records = []
    fig, axes = plt.subplots(2, 3, figsize=(15, 9))
    axes = axes.ravel()

    gt_cmap = plt.get_cmap('plasma').copy()
    gt_cmap.set_bad('lightgray')
    axes[0].imshow(np.ma.masked_invalid(gt_disp), cmap=gt_cmap, vmin=0, vmax=64,
        interpolation='nearest')
    axes[0].set_title("Ground Truth Disparity (gray = occluded)", fontweight='bold')
    axes[0].axis('off')

    best_disp = None

    for idx, (algo, bsize, ndisp) in enumerate(configs):
        raw_disp, runtime = compute_disparity(grayL, grayR, algo, bsize, ndisp)
        valid_ratio, rmse_val, bad_pixel_rate = evaluate_disparity(raw_disp, gt_disp,
            eval_margin)

        records.append({
            "Method": algo,
            "blockSize": bsize,
            "numDisparities": ndisp,
            "Valid Disparity (%)": round(valid_ratio, 2),
            "RMSE (px)": round(rmse_val, 2),
            "Bad-Pixel >2px (%)": round(bad_pixel_rate, 2),
            "Runtime (ms)": round(runtime, 2)
        })

        ax = axes[idx + 1]
        ax.imshow(raw_disp, cmap='plasma', vmin=0, vmax=64)
        ax.set_title(f"{algo} (b={bsize}, d={ndisp})\nValid: {valid_ratio:.1f}% | RMSE:
            {rmse_val:.2f}px")
        ax.axis('off')

        if algo == "SGBM" and bsize == 9 and ndisp == 64:
            best_disp = raw_disp

    plt.suptitle("Part B: Dense Disparity Map Comparison (BM vs. SGBM)", fontsize=14,
        fontweight='bold')
    plt.tight_layout()
    plt.savefig("results/part_b/disparity_comparison.png", dpi=180)
    plt.savefig("report_latex/figures/disparity_comparison.png", dpi=180)
    plt.close()

    df = pd.DataFrame(records)
    df.to_csv("results/part_b/quantitative_part_b.csv", index=False)
    print(df.to_string(index=False))

    parameter_sensitivity(grayL, grayR, gt_disp, eval_margin)

    # Reconstruct 3D Point Cloud
    h, w = grayL.shape
    f_len = 0.8 * w
    Q = np.float32([
        [1, 0, 0, -0.5 * w],
        [0, -1, 0, 0.5 * h],
        [0, 0, 0, -f_len],
        [0, 0, 1.0 / 0.12, 0]
    ])
```

```
    points_3d = cv2.reprojectImageTo3D(best_disp, Q)
    colors = cv2.cvtColor(imgL, cv2.COLOR_BGR2RGB)
    mask = (best_disp > 8.0) & (np.arange(w)[None, :] >= 64)

    out_points = points_3d[mask]
    out_colors = colors[mask]

    write_ply("results/part_b/reconstructed_cloud.ply", out_points[::4], out_colors[::4])

    fig = plt.figure(figsize=(13, 5.5))
    step = max(1, len(out_points) // 10000)
    pts_sub = out_points[::step]
    col_sub = out_colors[::step] / 255.0

    views = [
        (22, -55, "3D Point Cloud - Oblique Perspective View"),
        (68, -90, "3D Point Cloud - Top-Down Depth Profile")
    ]
    for i, (elev, azim, title) in enumerate(views):
        ax = fig.add_subplot(1, 2, i+1, projection='3d')
        ax.scatter(pts_sub[:, 0], pts_sub[:, 2], pts_sub[:, 1], c=col_sub, s=1.8, alpha=0.85)
        ax.set_xlabel("X (m)")
        ax.set_ylabel("Depth Z (m)")
        ax.set_zlabel("Y (m)")
        ax.view_init(elev=elev, azim=azim)
        ax.set_title(title, fontweight='bold')

    plt.tight_layout()
    plt.savefig("results/part_b/point_cloud_views.png", dpi=180)
    plt.savefig("report_latex/figures/point_cloud_views.png", dpi=180)
    plt.close()

if __name__ == "__main__":
    run_part_b()
```

A.3 Part C: Image Stitching

```
import os
import time
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

os.makedirs("results/part_c", exist_ok=True)
os.makedirs("report_latex/figures", exist_ok=True)

def create_4_overlapping_views(ref_path="data/reference_image.jpg"):
    img = cv2.imread(ref_path)
    # Resize to a wide high-res canvas (1200 x 650)
    canvas = cv2.resize(img, (1200, 650))
    H, W = canvas.shape[:2]

    # Extract 4 overlapping sub-views with subtle camera pitch/yaw perspective warps
    views = []
    crop_w, crop_h = 480, 560
    x_offsets = [0, 230, 460, 690]
    y_offsets = [40, 25, 45, 30]
    # Per-view exposure gains simulate auto-exposure changes between handheld shots
    gains = [1.00, 0.88, 1.10, 0.94]

    for i, (x0, y0) in enumerate(zip(x_offsets, y_offsets)):
        patch = canvas[y0:y0+crop_h, x0:x0+crop_w].copy()
        # Apply a slight perspective homography to simulate real handheld multi-view capture
        src_pts = np.float32([[0, 0], [crop_w, 0], [crop_w, crop_h], [0, crop_h]])
        shift = (-1)**i * 6.0
        dst_pts = np.float32([
            [max(0, shift), max(0, -shift)],
            [crop_w - max(0, -shift), max(0, shift)],
            [crop_w - max(0, shift), crop_h - max(0, -shift)],
            [max(0, -shift), crop_h - max(0, shift)]
```

```
        ])
        M = cv2.getPerspectiveTransform(src_pts, dst_pts)
        warped_view = cv2.warpPerspective(patch, M, (crop_w, crop_h),
            borderMode=cv2.BORDER_REFLECT)
        warped_view = cv2.convertScaleAbs(warped_view, alpha=gains[i])
        cv2.imwrite(f"data/view_{i+1}.png", warped_view)
        views.append(warped_view)

    # Save 4 input views figure
    fig, axes = plt.subplots(1, 4, figsize=(15, 4))
    for i, v in enumerate(views):
        axes[i].imshow(cv2.cvtColor(v, cv2.COLOR_BGR2RGB))
        axes[i].set_title(f"Overlapping View {i+1}", fontweight='bold')
        axes[i].axis('off')
    plt.suptitle("Part C: Four Overlapping Input Views from Reference Architectural Image",
        fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig("results/part_c/input_4_views.png", dpi=180)
    plt.savefig("report_latex/figures/input_4_views.png", dpi=180)
    plt.close()
    return views

def detect_and_match(img1, img2, detector_name="SIFT", ratio_thresh=0.75, ransac_thresh=4.0):
    if detector_name == "SIFT":
        detector = cv2.SIFT_create()
        norm = cv2.NORM_L2
    elif detector_name == "AKAZE":
        detector = cv2.AKAZE_create()
        norm = cv2.NORM_HAMMING
    else:
        detector = cv2.ORB_create(nfeatures=3000)
        norm = cv2.NORM_HAMMING

    kp1, des1 = detector.detectAndCompute(img1, None)
    kp2, des2 = detector.detectAndCompute(img2, None)

    matcher = cv2.BFMatcher(norm)
    raw_matches = matcher.knnMatch(des1, des2, k=2)

    good_matches = []
    for pair in raw_matches:
        if len(pair) == 2:
            m, n = pair
            if m.distance < ratio_thresh * n.distance:
                good_matches.append(m)

    pts1 = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    pts2 = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(pts2, pts1, cv2.RANSAC, ransac_thresh)
    inlier_mask = mask.ravel().astype(bool)

    inliers1 = pts1[inlier_mask]
    inliers2 = pts2[inlier_mask]

    proj_pts1 = cv2.perspectiveTransform(inliers2, H)
    reproj_errors = np.linalg.norm(inliers1 - proj_pts1, axis=2)
    rmse = float(np.sqrt(np.mean(reproj_errors**2)))

    return {
        "kp1": kp1, "kp2": kp2,
        "num_kp": len(kp1) + len(kp2),
        "initial_matches": len(good_matches),
        "inliers": int(np.sum(inlier_mask)),
        "inlier_ratio": (np.sum(inlier_mask) / max(1, len(good_matches))) * 100.0,
        "rmse": rmse,
        "H": H,
        "good_matches": good_matches,
        "mask": mask.ravel().tolist()
    }

def overwrite_blend(base_img, warped_img):
    mask_warp = (np.sum(warped_img, axis=2) > 0)[..., None]
```

```
    return np.where(mask_warp, warped_img, base_img)

def feather_blend(base_img, warped_img):
    mask_base = (np.sum(base_img, axis=2) > 0).astype(np.float32)
    mask_warp = (np.sum(warped_img, axis=2) > 0).astype(np.float32)

    dist_base = cv2.distanceTransform((mask_base * 255).astype(np.uint8), cv2.DIST_L2, 3)
    dist_warp = cv2.distanceTransform((mask_warp * 255).astype(np.uint8), cv2.DIST_L2, 3)

    total_weight = dist_base + dist_warp + 1e-6
    w1 = (dist_base / total_weight)[..., None]
    w2 = (dist_warp / total_weight)[..., None]

    blended = base_img.astype(np.float32) * w1 + warped_img.astype(np.float32) * w2
    return np.clip(blended, 0, 255).astype(np.uint8)

def multiband_blend(base_img, warped_img, levels=5):
    mask_base = np.sum(base_img, axis=2) > 0
    mask_warp = np.sum(warped_img, axis=2) > 0
    if not mask_base.any():
        return warped_img.copy()

    dist_base = cv2.distanceTransform(mask_base.astype(np.uint8), cv2.DIST_L2, 3)
    dist_warp = cv2.distanceTransform(mask_warp.astype(np.uint8), cv2.DIST_L2, 3)
    seam_mask = (dist_warp > dist_base).astype(np.float32)

    # Fill each image's empty area with the other so pyramid levels do not bleed black
        borders into the overlap
    a = np.where(mask_base[..., None], base_img, warped_img).astype(np.float32)
    b = np.where(mask_warp[..., None], warped_img, base_img).astype(np.float32)

    h, w = seam_mask.shape
    gauss_m, lap_a, lap_b = [seam_mask], [], []
    cur_a, cur_b = a, b
    for _ in range(levels):
        down_a, down_b = cv2.pyrDown(cur_a), cv2.pyrDown(cur_b)
        size = (cur_a.shape[1], cur_a.shape[0])
        lap_a.append(cur_a - cv2.pyrUp(down_a, dstsize=size))
        lap_b.append(cur_b - cv2.pyrUp(down_b, dstsize=size))
        gauss_m.append(cv2.pyrDown(gauss_m[-1]))
        cur_a, cur_b = down_a, down_b
    lap_a.append(cur_a)
    lap_b.append(cur_b)

    blended = None
    for la, lb, gm in reversed(list(zip(lap_a, lap_b, gauss_m))):
        level = la * (1.0 - gm[..., None]) + lb * gm[..., None]
        if blended is None:
            blended = level
        else:
            blended = cv2.pyrUp(blended, dstsize=(level.shape[1], level.shape[0])) + level

    union = (mask_base | mask_warp)[..., None]
    return np.where(union, np.clip(blended, 0, 255), 0).astype(np.uint8)

def stitch_4_images(views, detector_name="SIFT", ransac_thresh=4.0, blend="feather"):
    t_start = time.perf_counter()

    m01 = detect_and_match(views[1], views[0], detector_name, ransac_thresh=ransac_thresh)
    m12 = detect_and_match(views[1], views[2], detector_name, ransac_thresh=ransac_thresh)
    m23 = detect_and_match(views[2], views[3], detector_name, ransac_thresh=ransac_thresh)

    inlier_matches = [m for m, keep in zip(m12["good_matches"], m12["mask"]) if keep]
    match_vis = cv2.drawMatches(
        views[1], m12["kp1"], views[2], m12["kp2"],
        inlier_matches[:80], None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )
    if blend == "feather":
        cv2.imwrite(f"results/part_c/inliers_{detector_name}_r{int(ransac_thresh)}.png",
            match_vis)

    H_0_to_1 = m01["H"]
```

```
    H_1_to_1 = np.eye(3)
    H_2_to_1 = m12["H"]
    H_3_to_1 = m12["H"] @ m23["H"]
    homographies = [H_0_to_1, H_1_to_1, H_2_to_1, H_3_to_1]

    all_corners = []
    for img, H in zip(views, homographies):
        h, w = img.shape[:2]
        corners = np.float32([[0, 0], [0, h], [w, h], [w, 0]]).reshape(-1, 1, 2)
        warped_corners = cv2.perspectiveTransform(corners, H)
        all_corners.append(warped_corners)

    all_corners = np.concatenate(all_corners, axis=0)
    [xmin, ymin] = np.int32(all_corners.min(axis=0).ravel() - 0.5)
    [xmax, ymax] = np.int32(all_corners.max(axis=0).ravel() + 0.5)

    T_shift = np.array([[1, 0, -xmin], [0, 1, -ymin], [0, 0, 1]], dtype=np.float64)
    pano_w, pano_h = min(2400, xmax - xmin), min(1400, ymax - ymin)

    blend_fn = {"none": overwrite_blend, "feather": feather_blend, "multiband":
        multiband_blend}[blend]
    panorama = np.zeros((pano_h, pano_w, 3), dtype=np.uint8)
    t_blend = time.perf_counter()
    for img, H in zip(views, homographies):
        warped = cv2.warpPerspective(img, T_shift @ H, (pano_w, pano_h))
        panorama = blend_fn(panorama, warped)
    blend_time = (time.perf_counter() - t_blend) * 1000

    total_time = (time.perf_counter() - t_start) * 1000
    suffix = "" if blend == "feather" else f"_{blend}"
    cv2.imwrite(f"results/part_c/panorama_{detector_name}_r{int(ransac_thresh)}{suffix}.png",
        panorama)

    pair_stats = [m01, m12, m23]
    return {
        "Detector": detector_name,
        "RANSAC Thresh (px)": ransac_thresh,
        "Blending": blend,
        "Total Keypoints": sum(s["num_kp"] for s in pair_stats),
        "Initial Matches": sum(s["initial_matches"] for s in pair_stats),
        "Inliers": sum(s["inliers"] for s in pair_stats),
        "Inlier Ratio (%)": round(np.mean([s["inlier_ratio"] for s in pair_stats]), 2),
        "Mean RMSE (px)": round(np.mean([s["rmse"] for s in pair_stats]), 3),
        "Blending Time (ms)": round(blend_time, 2),
        "Stitching Time (ms)": round(total_time, 2)
    }, match_vis, panorama

def run_part_c():
    views = create_4_overlapping_views()
    configs = [
        ("SIFT", 2.0),
        ("SIFT", 4.0),
        ("ORB", 2.0),
        ("ORB", 4.0),
        ("AKAZE", 4.0)
    ]
    records = []
    vis_dict = {}
    for det, r_thresh in configs:
        res, match_vis, pano = stitch_4_images(views, detector_name=det,
            ransac_thresh=r_thresh)
        records.append(res)
        if r_thresh == 4.0:
            vis_dict[det] = (match_vis, pano, res)

    df = pd.DataFrame(records)
    df.to_csv("results/part_c/quantitative_part_c.csv", index=False)
    print(df.to_string(index=False))

    plot_keypoints(views[1])
    compare_blending(views)

    # Create comparison figure for report
```

```
    fig, axes = plt.subplots(3, 2, figsize=(15, 11))
    for idx, det in enumerate(["SIFT", "ORB", "AKAZE"]):
        m_vis, pano_img, st = vis_dict[det]
        axes[idx, 0].imshow(cv2.cvtColor(m_vis, cv2.COLOR_BGR2RGB))
        axes[idx, 0].set_title(f"{det} Inlier Matches (Inliers: {st['Inliers']}, Ratio:
            {st['Inlier Ratio (%)']:.1f}%)", fontweight='bold')
        axes[idx, 0].axis('off')

        axes[idx, 1].imshow(cv2.cvtColor(pano_img, cv2.COLOR_BGR2RGB))
        axes[idx, 1].set_title(f"{det} Stitched 4-View Panorama (RMSE: {st['Mean RMSE
            (px)']:.3f}px, Time: {st['Stitching Time (ms)']:.1f}ms)", fontweight='bold')
        axes[idx, 1].axis('off')

    plt.suptitle("Part C: Multi-Detector Feature Matching & 4-View Panorama Comparison",
        fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.savefig("results/part_c/stitching_comparison.png", dpi=180)
    plt.savefig("report_latex/figures/stitching_comparison.png", dpi=180)
    plt.close()

def plot_keypoints(view):
    detectors = {
        "SIFT": cv2.SIFT_create(),
        "ORB": cv2.ORB_create(nfeatures=3000),
        "AKAZE": cv2.AKAZE_create()
    }
    fig, axes = plt.subplots(1, 3, figsize=(15, 6))
    for ax, (name, det) in zip(axes, detectors.items()):
        kps = det.detect(view, None)
        vis = view.copy()
        for kp in kps:
            cv2.circle(vis, tuple(np.int32(kp.pt)), 2, (0, 255, 0), -1)
        ax.imshow(cv2.cvtColor(vis, cv2.COLOR_BGR2RGB))
        ax.set_title(f"{name}: {len(kps)} keypoints (View 2)", fontweight='bold')
        ax.axis('off')
    plt.suptitle("Part C: Keypoint Detection Comparison", fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig("results/part_c/keypoints_comparison.png", dpi=180)
    plt.savefig("report_latex/figures/keypoints_comparison.png", dpi=180)
    plt.close()

def compare_blending(views, detector_name="SIFT", ransac_thresh=4.0):
    records, panos = [], {}
    for blend in ["none", "feather", "multiband"]:
        res, _, pano = stitch_4_images(views, detector_name, ransac_thresh, blend=blend)
        records.append(res)
        panos[blend] = pano

    df = pd.DataFrame(records)
    df.to_csv("results/part_c/blending_comparison.csv", index=False)
    print(df.to_string(index=False))

    # Crop around the View 2 / View 3 overlap where the exposure jump (0.88 -> 1.10) is
        largest
    h, w = panos["none"].shape[:2]
    cx, half = int(w * 0.5), int(w * 0.12)
    titles = {"none": "No blending (overwrite)", "feather": "Feather (distance transform)",
              "multiband": "Laplacian pyramid (5 levels)"}
    fig, axes = plt.subplots(2, 3, figsize=(16, 8.5), gridspec_kw={"height_ratios": [1, 1.3]})
    for col, blend in enumerate(["none", "feather", "multiband"]):
        pano = cv2.cvtColor(panos[blend], cv2.COLOR_BGR2RGB)
        axes[0, col].imshow(pano)
        axes[0, col].axvspan(cx - half, cx + half, color='yellow', alpha=0.15)
        axes[0, col].set_title(f"{titles[blend]}\nBlend time: {records[col]['Blending Time
            (ms)']:.0f} ms",
                               fontweight='bold')
        axes[1, col].imshow(pano[:, cx - half:cx + half])
        axes[1, col].set_title("Zoom on View 2 / View 3 overlap", fontsize=10)
        for ax in axes[:, col]:
            ax.axis('off')
    plt.suptitle("Part C: Blending Comparison under Exposure Differences (SIFT, RANSAC 4 px)",
                 fontsize=14, fontweight='bold')
    plt.tight_layout()
```

```
    plt.savefig("results/part_c/blending_comparison.png", dpi=180)
    plt.savefig("report_latex/figures/blending_comparison.png", dpi=180)
    plt.close()

if __name__ == "__main__":
    run_part_c()
```